

Project Documentation

AlgoPilot (Working Title)

An intelligent DSA practice platform that recommends personalized coding problems based on a user's strengths, weaknesses, solving history, and learning goals.

1. Vision

Most developers use platforms like LeetCode by manually selecting problems from lists such as NeetCode 150 or Blind 75. While these curated lists are excellent, they are static and do not adapt to each individual's learning progress.

AlgoPilot aims to become a personalized learning companion that continuously analyzes a user's performance and recommends the most valuable problem to solve next.

Instead of asking:

"What should I solve today?"

the platform answers it automatically.

2. Goals

Primary Goals

- Personalized daily recommendations
- Track learning progress
- Identify weak topics
- Improve long-term retention through spaced repetition
- Help users prepare for coding interviews

Secondary Goals

- Analytics dashboard
 - Interview roadmaps
 - Study streaks
 - Progress tracking
 - Browser extension
 - AI learning assistant
-

3. Target Users

Students

Preparing for internships or placements.

Software Engineers

Preparing for interviews at top companies.

Competitive Programmers

Wanting structured revision.

Self Learners

Learning DSA efficiently.

4. MVP Scope

The first version will focus only on curated problem sheets.

Supported Sheets

- NeetCode 150
- Blind 75
- Grind 169 (later)
- Top Interview 150 (later)
- Striver SDE Sheet (later)

The MVP will not support the entire LeetCode catalog.

5. Core Features

Authentication

Users can create an account and securely log in.

Future support:

- Google
 - GitHub
-

Problem Import System

Instead of manually inserting problems, the application imports problems from configurable sources.

Architecture

Source → Normalizer → Database

The import process must be idempotent so running it multiple times does not create duplicate records.

Problem Library

Each problem stores:

- Title
 - Difficulty
 - URL
 - Topics
 - Acceptance Rate
 - Premium Status
 - Problem Set
-

User Progress

Track

- Solved problems
 - Attempted problems
 - Review history
 - Confidence score
 - Time spent
 - Number of attempts
-

Daily Recommendations

Every day the platform generates personalized recommendations.

The recommendation engine considers

- Weak topics
 - Unsolved problems
 - Previously struggled problems
 - Review schedule
 - Difficulty progression
-

Dashboard

Displays

- Daily recommendations
 - Progress statistics
 - Weak topics
 - Streak
 - Topic mastery
 - Recent activity
-

Analytics

Examples

- Problems solved
 - Problems remaining
 - Easy/Medium/Hard distribution
 - Average solve time
 - Weekly progress
 - Monthly progress
-

Notes

Every problem supports personal notes.

Examples

- Pattern learned
 - Mistakes made
 - Edge cases
 - Complexity analysis
-

Review System

Implements spaced repetition.

Review intervals

- 1 day
- 3 days
- 7 days
- 14 days
- 30 days
- 90 days

6. Future Features

Browser Extension

Features

- Show today's recommendations inside LeetCode
 - Save progress
 - One-click notes
 - Recommend similar problems
-

AI Coach

The AI should never directly reveal the solution.

Instead it provides

- Hints
 - Pattern guidance
 - Complexity suggestions
 - Edge case reminders
-

Interview Planner

Generate study plans such as

30-Day Interview Plan

60-Day Placement Plan

90-Day FAANG Preparation

Company Preparation

Examples

Amazon

↓

Recommended problems

↓

Progress



Weak areas

7. Technical Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

Database

- PostgreSQL

ORM

- Drizzle ORM

Authentication

- Better Auth

Validation

- Zod

Charts

- Recharts

Deployment

- Vercel
-

8. High-Level Architecture

Client



Next.js App

↓

Server Actions / Route Handlers

↓

Business Services

↓

Database

↓

PostgreSQL

Business Services include

- Recommendation Engine
- Analytics
- Problem Sources
- Review Scheduler

9. Project Structure

```
app/  
components/  
db/  
lib/  
types/  
scripts/  
public/
```

Important folders

lib/

recommendation/

analytics/

sources/

utils/

db/

schema/

migrations/

10. Database (Initial)

Tables

Problems

Topics

ProblemTopics

ProblemSets

ProblemSetProblems

Future Tables

Users

UserProblems

Recommendations

Attempts

Reviews

Notes

LearningPaths

11. Recommendation Engine

Version 1 uses a rule-based scoring system.

Example

Weak Topic

+40

Not Reviewed Recently

+25

Previously Struggled

+20

Low Confidence

+15

Highest scoring problems become today's recommendations.

Future versions may use machine learning.

12. Development Roadmap

Phase 1

Project setup

Database

Problem import

Phase 2

Authentication

Dashboard

Problem library

Phase 3

Progress tracking

Recommendations

Analytics

Phase 4

Review system

Notes

Learning paths

Phase 5

Browser extension

AI Coach

Interview planner

13. Design Principles

- Keep business logic separate from UI.
 - Use TypeScript throughout the project.
 - Prefer composition over duplication.
 - Keep database normalized.
 - Make importers modular.
 - Design APIs to be reusable.
 - Build for extensibility without unnecessary complexity.
-

14. Success Metrics

Technical

- Import curated problem sets successfully.
- Generate personalized recommendations.
- Maintain fast response times.
- Zero duplicate imports.

User

- Daily active usage.
 - Recommendation completion rate.
 - Practice streak.
 - Increase in topic mastery.
 - User retention.
-

15. Long-Term Vision

AlgoPilot should evolve beyond a simple problem tracker.

The ultimate vision is to become a complete interview preparation platform that understands how each user learns, adapts recommendations over time, and acts as an intelligent mentor throughout their coding interview journey.

The platform should eventually support multiple coding platforms, AI-assisted learning, collaborative study groups, custom learning paths, company-specific preparation, and advanced analytics while maintaining a personalized learning experience for every user.